Lecture 24: Cryptography and Security

From Primitives to TLS

EC 441 — Introduction to Computer Networking
Boston University, Spring 2026

Tuesday, April 28, 2026

Learning Objectives

By the end of this lecture you should be able to:

1. State the four core security requirements — *confidentiality, integrity, authentication, non-repudiation* — and name a cryptographic primitive that addresses each.
2. Distinguish symmetric from asymmetric cryptography, and explain when each is used in practice.
3. Derive the RSA algorithm from Euler's theorem and work a numerical example end to end.
4. Explain how a hash function differs from encryption, and list the three security properties.
5. Describe the PKI trust model and explain how a browser verifies a TLS certificate chain.
6. Walk through a TLS 1.3 handshake and identify where each primitive from the lecture is used.

Framing

This is the last content lecture of the semester. L23 ended with the observation that every protocol we had studied runs in cleartext by default — *"everything you watched today, tcpdump could read."* L24 is the answer: the cryptographic building blocks, and how TLS assembles them into the security the modern web runs on.

Actor naming. We use α (alpha) for the sender, β (beta) for the receiver, and τ (tau) for the adversary throughout. These are the three characters in every crypto diagram.

Structure. Session 1 is mathematical: basic primitives plus RSA with actual math. Session 2 is practical: how the primitives compose into real security protocols (MACs, signatures, PKI, TLS).

1 Threat Model

α wants to send a message to β. τ sits somewhere on the path. What can τ do?

Attack	Threat	Requirement
Eavesdrop	read the message	confidentiality
Modify	change bits mid-flight	integrity
Impersonate	pretend to be peer	authentication
Replay	resend old captured msg	freshness
Deny later	"I never sent that"	non-repudiation

Building security is the process of addressing these threats one at a time, each with an appropriate cryptographic primitive. This table is the spine of the lecture — every subsequent section maps to a row here.

2 Symmetric Cryptography

Setup. α and β share a secret key k. The same key is used to encrypt and decrypt:

$$c = E_k(m) \qquad m = D_k(c).$$

Workhorse: AES. The Advanced Encryption Standard (Rijndael, NIST FIPS 197, 2001) operates on 128-bit blocks with 128, 192, or 256-bit keys. Hardware-accelerated on essentially every modern CPU (the AES-NI instructions), so AES-GCM runs at multiple GB/s per core on a laptop — effectively free.

Block cipher modes. The raw block cipher only handles a single 128-bit block. To encrypt longer messages, we compose the block cipher with a *mode of operation*:

- **ECB (Electronic Codebook).** Encrypt each block independently. Patterns in the plaintext leak through (the famous "ECB penguin" image demonstrates this). **Never use.**
- **CBC (Cipher Block Chaining).** XOR each block with the previous ciphertext; requires an IV. Provides confidentiality but not integrity. Legacy.
- **GCM (Galois Counter Mode).** Authenticated encryption — provides confidentiality *and* integrity in one primitive. This is what TLS 1.3 uses.

The key distribution problem

Symmetric crypto requires a shared key. How do α and β agree on k when τ is watching the channel? You cannot just send the key over the same channel you are trying to protect. Three answers build up over the next few sections: pre-shared keys (works, doesn't scale), asymmetric crypto to transport the key (the classic TLS 1.2 pattern), and Diffie–Hellman key agreement (the TLS 1.3 pattern).

3 Hash Functions

A hash function

$$h : \{0,1\}^* \to \{0,1\}^n$$

maps arbitrary-length input to a fixed-length digest. SHA-256 has $n = 256$; SHA-512 has $n = 512$.

Not encryption. There is no key. There is no way to "decrypt." A hash is a one-way compression of data into a fingerprint.

Three security properties.

1. **Preimage resistance.** Given $y = h(m)$, infeasible to find any m' with $h(m') = y$. (Cannot invert.)
2. **Second-preimage resistance.** Given m, infeasible to find $m' \neq m$ with $h(m') = h(m)$.
3. **Collision resistance.** Infeasible to find any pair $m \neq m'$ with $h(m) = h(m')$.

By the birthday paradox, collision resistance on an n-bit hash gives only $\sim 2^{n/2}$ security; SHA-256 gives $\sim 2^{128}$ collision resistance, enough for the foreseeable future.

Uses. Integrity checks (publish a file and its SHA-256; downloaders verify), password storage (store $h(\text{salt} \,\|\, \text{password})$, never the password itself), commitment schemes ("here's $h(m)$ — I'll reveal m later"), blockchain block linking, and as a building block for MACs and digital signatures below.

Hash function properties

Script: `demo_hash_l24.py`
Demonstrates:

- Fixed output size regardless of input size (0 B, 1 B, 1 MB inputs all produce 256-bit outputs).
- Avalanche effect: adding one character to a message flips ∼50% of output bits.
- A toy commitment scheme: α publishes $h(\text{nonce}\|\text{bid})$ and can only open it to the original bid.

4 Asymmetric Cryptography and RSA

Each party holds a **key pair**: a public key K_{pub} and a private key K_{priv}. The keys are mathematically linked; knowing the public key does not reveal the private key. Two fundamental operations:

- **Encryption.** Anyone with β's public key can encrypt a message that only β (holder of the matching private key) can decrypt. Solves confidentiality.
- **Signing.** The holder of a private key produces a signature that anyone with the public key can verify. Solves authentication, integrity, and non-repudiation.

The asymmetry: encryption uses the *receiver's* public key; signing uses the *sender's* private key.

4.1 Number theory

Two classical results

Euler's totient. $\varphi(n)$ is the number of integers in $[1, n]$ coprime to n. For a prime p, $\varphi(p) = p - 1$. For $n = pq$ with distinct primes, $\varphi(n) = (p-1)(q-1)$.
Euler's theorem. If $\gcd(a, n) = 1$,

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

(Fermat's little theorem is the prime-n special case.)

RSA is a three-line corollary of Euler's theorem.

4.2 RSA key generation

1. Choose two large primes p, q (1024 bits each, in practice).
2. Compute $n = pq$ and $\varphi(n) = (p-1)(q-1)$.
3. Choose a public exponent e with $\gcd(e, \varphi(n)) = 1$. In practice almost always $e = 65537$.
4. Compute $d = e^{-1} \pmod{\varphi(n)}$ via the extended Euclidean algorithm.

Public key: (n, e). **Private key:** (n, d).

4.3 Encryption and decryption

Encode the message as an integer m with $0 \le m < n$.

$$\text{Encrypt:} \quad c = m^e \bmod n \qquad \text{Decrypt:} \quad m = c^d \bmod n$$

4.4 Why it works

We chose $ed \equiv 1 \pmod{\varphi(n)}$, so $ed = 1 + k\varphi(n)$ for some integer k. For m coprime to n:

$$c^d = m^{ed} = m \cdot \left(m^{\varphi(n)}\right)^k \equiv m \cdot 1^k \equiv m \pmod{n}$$

by Euler's theorem. (The case $\gcd(m, n) > 1$ also works, via the Chinese Remainder Theorem, but is rarely relevant for properly-sized RSA.)

4.5 Why it's secure

An attacker who knows (n, e) and wants d needs $\varphi(n) = (p-1)(q-1)$, which requires factoring n into its two prime factors. Factoring a 2048-bit semiprime is believed to take $\sim 2^{112}$ operations with the best known classical algorithms — well beyond feasibility. Shor's algorithm on a sufficiently large quantum computer factors in polynomial time, which is why post-quantum cryptography is an active area.

Worked RSA example

Demo script: `demo_rsa_math_l24.py`.
Key generation. $p = 11$, $q = 13$, so

$$n = 11 \cdot 13 = 143, \qquad \varphi(n) = 10 \cdot 12 = 120.$$

Pick $e = 7$ (coprime to 120). Solve $7d \equiv 1 \pmod{120}$ to get $d = 103$. (Check: $7 \cdot 103 = 721 = 6 \cdot 120 + 1$.)
Encrypt $m = 9$:

$$c = 9^7 \bmod 143 = 4{,}782{,}969 \bmod 143 = 48.$$

Decrypt $c = 48$:

$$m = 48^{103} \bmod 143 = 9.$$

Run the demo script and change numbers. The math fits on one page.

4.6 Signing

RSA signing uses the same math with the key roles swapped:

$$\text{Sign:} \quad s = h(m)^d \bmod n \qquad \text{Verify:} \quad s^e \bmod n \stackrel{?}{=} h(m).$$

The signer applies the private exponent; any verifier applies the public exponent. We sign a hash of the message (not the message itself) for two reasons: efficiency (one exponentiation signs any length) and security (textbook RSA on raw m has known forgery attacks via its multiplicativity; RSA-PSS padding avoids them).

5 Diffie–Hellman Key Exchange

A short but essential beat. Asymmetric encryption *transports* a key; Diffie–Hellman *agrees on* one without ever transmitting it.

Public parameters. A large prime p and a generator g of $(\mathbb{Z}/p\mathbb{Z})^*$. Anyone can know these.

Protocol.

1. α picks private a; sends $A = g^a \bmod p$.
2. β picks private b; sends $B = g^b \bmod p$.
3. α computes $B^a = g^{ab} \bmod p$.
4. β computes $A^b = g^{ab} \bmod p$.

α and β now share the secret $g^{ab} \bmod p$ without ever sending it. τ sees g^a and g^b on the wire; to recover g^{ab}, τ must solve the *discrete logarithm problem* — believed hard.

Ephemeral DH and forward secrecy

Every modern TLS connection uses *ephemeral* Diffie–Hellman (usually on an elliptic curve). The server's long-term RSA or ECDSA key is used *only* to sign the DH public share, not to transport the session key.
Consequence: even if the server's long-term private key leaks tomorrow, traffic recorded today cannot be decrypted — the session key was derived from ephemeral values that were discarded. This property is called **forward secrecy**, and it is why TLS 1.3 removed the old RSA key-transport mode entirely.

6 Mapping Primitives to Requirements

With all the primitives in place, the threat-model table from Section 1 becomes concrete:

Requirement	Primitive	Notes
Confidentiality	AES-GCM or similar AEAD	fast, shared key
Integrity	MAC or AEAD	hash + key
Authentication (peer)	signature or MAC	who is this
Authentication (message)	MAC	someone with the key made this
Non-repudiation	signature *only*	asymmetric
Freshness	nonces, timestamps	protocol level

Why signatures for non-repudiation, not MACs

A MAC proves *somebody who knew the key* made the message — but both α and β know the key, so neither can prove to a third party that the other one made it. A signature is made with a private key that only one party holds, so the third party can verify it too. This is why legal and financial systems use signatures, not MACs.

7 MACs and Digital Signatures

7.1 MACs — Message Authentication Codes

Given a shared key k, a MAC proves:

- The message has not been modified (integrity).
- It came from someone who knows k (authentication).

HMAC (RFC 2104) is the standard construction:

$$\mathrm{HMAC}_k(m) = h\big((k \oplus \mathrm{opad}) \,\|\, h((k \oplus \mathrm{ipad}) \,\|\, m)\big),$$

where h is a hash function and `ipad`, `opad` are fixed constants. The two-level hash defends against extension attacks on naive $h(k\|m)$.

HMAC-SHA-256 is a common default. In modern TLS, MACs are subsumed by AEAD (AES-GCM), which rolls encryption and authentication into one primitive.

7.2 Digital signatures

$$s = \text{Sign}_{K_{\text{priv}}}(h(m)), \qquad \text{Verify}_{K_{\text{pub}}}(s, h(m)) \in \{T, F\}.$$

Signatures provide integrity, peer authentication, *and* non-repudiation in one primitive — the last because only the private key holder could have produced the signature, and this is verifiable by any third party.

Modern alternatives to RSA. Elliptic-curve signatures are much smaller for the same security:

- Ed25519 signature: ∼64 bytes.
- RSA-2048 signature: 256 bytes.

Most new systems default to Ed25519 or ECDSA.

8 Public Key Infrastructure

We now have digital signatures. But how does β's browser know that a public key presented by someone claiming to be `bank.com` *actually* belongs to the bank?

The trust problem

A public key by itself is just bits. It says nothing about whose key it is. An attacker τ can generate their own key pair and claim to be anyone.

The answer: certificates. A certificate binds a public key to an identity by having a trusted third party sign the binding.

$$\text{cert} = \big(\text{identity}, K_{\text{pub}}, \text{validity}, \ldots\big) + \text{signature by a CA}.$$

A **Certificate Authority** (CA) is a trusted third party. Browsers and operating systems ship a **root store** — a preinstalled list of root CA public keys.

The chain of trust. In practice, certificates chain:

$$\text{leaf cert} \xrightarrow{\text{signed by}} \text{intermediate CA} \xrightarrow{\text{signed by}} \text{root CA}.$$

β's browser walks the chain from leaf to a root in its store. Roots are few (hundreds globally), intermediates are many, leaves are one per domain (or wildcarded).

8.1 Let's Encrypt

Free, automated (ACME protocol), launched 2016. HTTPS share of web traffic went from ∼30% in 2015 to ∼95% today, largely because Let's Encrypt made certs free and automatic. It changed the economics of "HTTPS by default."

8.2 When CAs go bad

A CA compromise is a catastrophic trust failure. Two case studies:

- **DigiNotar (2011).** Compromised by an attacker who issued fraudulent `*.google.com` certs used to intercept Iranian users' Gmail. Removed from every major root store; the company went bankrupt within weeks.
- **Symantec (2017–18).** Google progressively distrusted Symantec's CA hierarchy after a series of mis-issuance incidents. Symantec eventually sold its CA business to DigiCert.

These are real — and they show the recovery process works: compromised CAs get removed, and the rest of the system keeps functioning.

Certificate pinning. Apps (and sometimes browsers, for a limited set of domains) can "pin" to a specific expected certificate or public key rather than trusting the whole root store. Defense in depth at the cost of operational fragility — a pin in code means the cert cannot be rotated without an app update.

Inspect a real cert chain

Script: `demo_openssl_client_l24.sh`

```
openssl s_client -connect bu.edu:443 -servername bu.edu < /dev/null
openssl s_client -connect expired.badssl.com:443 \
    -servername expired.badssl.com
```

Look in the output for:

- The `depth` counter: 0 is the leaf; higher is intermediate CAs, up to a root in your local store.
- `subject` and `issuer` lines.
- `Server Temp Key` — the ephemeral DH share for this session (forward secrecy in action).
- `Cipher: TLS_AES_256_GCM_SHA384` — AES-GCM and a SHA-384-based KDF. The whole L24 primitive toolkit in one line.

9 TLS

TLS 1.3 (RFC 8446, 2018) is where every primitive from this lecture composes into a real protocol. The handshake is a dance, but every step uses something we have already covered.

9.1 The TLS 1.3 handshake

1. **ClientHello.** α sends its list of supported ciphers and an ephemeral DH public share g^a.
2. **ServerHello.** β picks a cipher and sends:
   - its own ephemeral DH public share g^b,

- its certificate chain,
- a signature over the handshake transcript using β's long-term private key.

3. **Client verifies.** α verifies β's certificate chain against its root store, then verifies β's signature — confirming β holds the private key for the cert.
4. **Both derive session keys.** Both sides compute g^{ab}, run it through a hash-based key-derivation function (HKDF), and produce the symmetric AES-GCM keys used for the rest of the session.
5. **Application data.** Encrypted with AES-GCM. Each record gets confidentiality and integrity from the AEAD primitive.

9.2 Every primitive, one handshake

Step	Primitive	Covered in
ephemeral key share	Diffie–Hellman	§Diffie–Hellman
cert chain verification	signatures + PKI	§Signatures, §PKI
handshake authenticity	signature on transcript	§Signatures
bulk data encryption	AES-GCM (AEAD)	§Symmetric
bulk data integrity	AES-GCM (AEAD)	§Symmetric
session key derivation	HKDF (hash-based)	§Hash

The division of labor

Asymmetric crypto is used once, at the handshake (tens of milliseconds per session): ephemeral DH for key agreement, RSA or ECDSA for the signature over the transcript.
Symmetric crypto is used everywhere else (GB/s, essentially free): AES-GCM on every record.
That division is the whole design, and it is why modern TLS has essentially no throughput penalty over plaintext.

9.3 Loop back to L23: QUIC

Recall from L23 that QUIC is HTTP/3's transport, over UDP, with TLS built in. "Built in" means QUIC integrates the TLS 1.3 handshake into the transport-layer handshake itself:

- One round-trip establishes the connection *and* the crypto.
- 0-RTT resumption sends application data in the first packet on return visits to a server.
- Streams are encrypted independently, and even packet headers are partially encrypted.

The promise from L23, delivered: QUIC closes the full loop. Every layer, encrypted by default, in one round-trip.

10 Closing

The course arc, in three sentences:

1. **L1–L9.** What bits mean, how they travel, how they survive noise, how a local network works.
2. **L13–L22.** How packets get across the world, how reliability is built on best-effort, and the tools that let you see and touch it.
3. **L23–L24.** What people actually build on top — HTTP, DNS, QUIC, TLS — and the cryptography that makes any of it safe.

What to study next. Not on any exam, just seeds. IPv6 deployment and the long migration; BGP and the economics of internet routing; software-defined networking (SDN, OpenFlow, P4); net neutrality, surveillance, and privacy-enhancing technologies; post-quantum cryptography — the next decade of TLS; high-throughput systems (DPDK, RDMA, kernel bypass). You now have the vocabulary to read anything in any of these areas.

What's next. L25 — Thursday, April 30 — project demo day. Show us what you built.

Acronyms

Acronym	Expansion
ACME	Automatic Certificate Management Environment
AES	Advanced Encryption Standard
AEAD	Authenticated Encryption with Associated Data
CA	Certificate Authority
CBC	Cipher Block Chaining
DH	Diffie–Hellman
ECB	Electronic Codebook
ECDSA	Elliptic Curve Digital Signature Algorithm
FIPS	Federal Information Processing Standard
GCM	Galois/Counter Mode
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
MAC	Message Authentication Code
NIST	National Institute of Standards and Technology
PKI	Public Key Infrastructure
PSK	Pre-Shared Key
QUIC	Quick UDP Internet Connections
RSA	Rivest–Shamir–Adleman
SHA	Secure Hash Algorithm
TLS	Transport Layer Security

References

- Wikipedia: RSA (cryptosystem)

- Wikipedia: Advanced Encryption Standard
- Wikipedia: Diffie-Hellman key exchange
- Wikipedia: HMAC
- Wikipedia: Public key infrastructure
- Wikipedia: Transport Layer Security
- Wikipedia: Elliptic-curve cryptography
- RFC 8446 — TLS 1.3
- RFC 2104 — HMAC
- RFC 8017 — RSA (PKCS #1 v2.2)
- Let's Encrypt — the CA that made HTTPS default.